

C++ Programming Textbook Notes

Module: Base Class Pointers, Compile-Time Binding vs. Runtime Binding, & Virtual Functions

1. THE BASE CLASS POINTER RULE

Fundamental Rule

Conceptual Foundation

In C++ Object-Oriented Programming, pointer assignment across inheritance hierarchies follows a fundamental rule:

The Fundamental Rule of Pointer Assignment

A Base Class Pointer can point to an object of its Base Class OR to an object of ANY of its Descendant (Derived/Child) Classes.

However, the converse is NOT true: A Derived Class Pointer CANNOT point to a Base Class Object without explicit, unsafe casting.

Valid vs. Invalid Pointer Compatibility

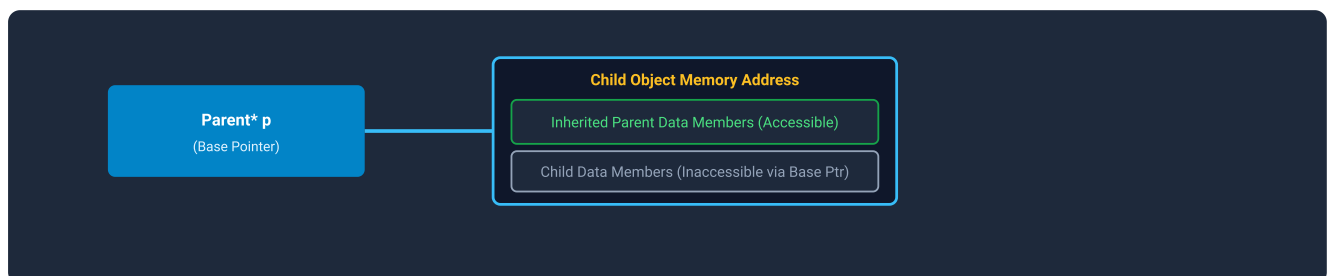
Because a derived class object contains all member attributes inherited from its parent class (plus its own specialized additions), treating a derived object as a base class instance is type-safe. Conversely, a base class object lacks the specialized members of a derived class, making derived pointers pointing to base objects illegal.

✓ Legal & Safe

```
Parent* p; Child cobj; p = &cobj; // VALID: Base pointer -> Child obj
```

✗ Illegal & Unsafe

Memory Pointer Model



Calling Functions Through Pointers: Syntax Rule

When accessing class members directly via object variables, C++ uses the dot operator (.). However, when accessing functions or data members through a pointer variable, C++ requires the arrow operator (->):

```
Parent obj; Parent* p = &obj; obj.show(); // Object Dot Syntax p->show(); // Pointer Arrow Syntax
```

2. COMPILE-TIME (EARLY) BINDING DEMONSTRATION

Static Behavior

How Non-Virtual Function Calls are Resolved

By default in C++, function calls made through pointers are resolved strictly based on the **datatype of the pointer itself**, rather than the actual object type stored at that memory address. This is called **Compile-time Binding (or Early Binding)**.

Code Demonstration: Non-Virtual Method Binding

```
#include <iostream> using namespace std; class Parent { public: void show() { cout << "This is Parent class show()"; }
```

Why Did Case 2 Output the Parent Method?

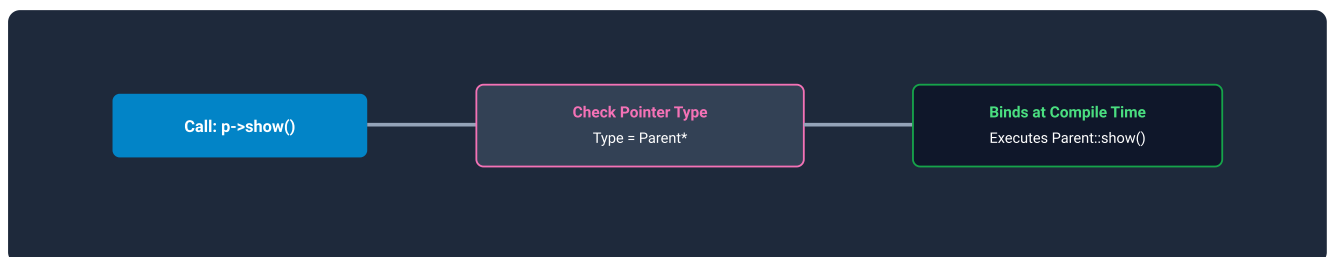
Even though p holds the address of a Child object (&obj2), calling p->show() still outputs This is Parent class show().

The Core Compiler Rule (Without virtual):

- **Direct Object Calls:** Function calls are decided by the *Object Type*.
- **Pointer Calls (Without virtual):** Function calls are decided by the *Pointer Type*.

Since pointer p was declared as Parent*, the compiler binds the call statically at compile-time to Parent::show().

Compile-Time Call Resolution Flow



Introducing Virtual Functions

To resolve function calls at runtime based on the **actual object stored in memory** rather than the pointer type, C++ introduces the `virtual` keyword.

Definition: Virtual Function

A **Virtual Function** is a member function declared inside a base class using the keyword `virtual`. It instructs the compiler to ensure **Late Binding (Run-time Binding)** so that function calls are resolved at runtime based on the actual target object.

Code Demonstration: Virtual Function Implementation

```
#include <iostream> using namespace std; class Parent { public: // Marking function as virtual
```

Program Output Comparison

Pointer Assignment	Without virtual (Early Binding)	With virtual (Late Binding)
p = &obj1; (Parent Obj)	This is Parent class show()	This is Parent class show()
p = &obj2; (Child Obj)	This is Parent class show()	This is Child class show()

Compiler Decision Mechanism

When parsing a function call made through a base pointer, the compiler inspects the base class declaration:

- **If NOT `virtual`:** Generates a direct assembly jump instruction to the base function address at compile time.
- **If marked `virtual`:** Generates dynamic lookup instructions to read the object's runtime table (vtable) when executed.

Compile-Time Binding vs. Run-Time Binding

Parameter	Compile-Time / Early Binding	Run-Time / Late Binding
Definition	Function call is linked to code block at compile time.	Function call is linked to code block dynamically at runtime.
Mechanism	Resolved based on the Pointer/Reference Datatype .	Resolved based on the Actual Object Type in memory.
Keyword	Default behavior (no keyword needed).	Requires the <code>virtual</code> keyword in the base class.
Execution Speed	Faster execution (direct assembly call instruction).	Slight overhead due to vtable pointer lookup.
Flexibility	Rigid and non-polymorphic.	Highly flexible and enables true Runtime Polymorphism.

Rules of Thumb for C++ Developers**Golden Rules to Remember**

1. **Base Pointer Assignment:** A base class pointer can point to any derived object, but a derived pointer cannot point to a base object.
2. **Arrow Syntax:** Always use `ptr->member` when invoking functions through a pointer.
3. **Virtual Keyword Inheritance:** Once a function is declared `virtual` in a base class, it remains virtual across all child classes automatically.
4. **Virtual Destructor Requirement:** Always declare a virtual destructor in a base class that contains virtual functions (`virtual ~Parent() = default;`) to prevent memory leaks when deleting derived objects via base pointers.

Summary Checklist**Without `virtual`**

- Static Binding
- Call decided by Pointer Type
- Fast, but lacks dynamic flexibility

With `virtual`

- Dynamic / Late Binding
- Call decided by Actual Object Type
- Key to Runtime Polymorphism

Key Takeaway: Runtime Polymorphism in C++ is achieved using **Base Class Pointers** combined with **Virtual Functions**. This design pattern allows systems to process heterogeneous derived classes uniformly through a clean base interface.